

Finger Control

Finger Control

1. Course Content
2. Program Startup
3. Core Code Analysis

1. Course Content

- Run example programs, use the mediapipe framework to detect fingers, then calculate the angle formed by the thumb fingertip, wrist joint, and index fingertip to change the image processing effects.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

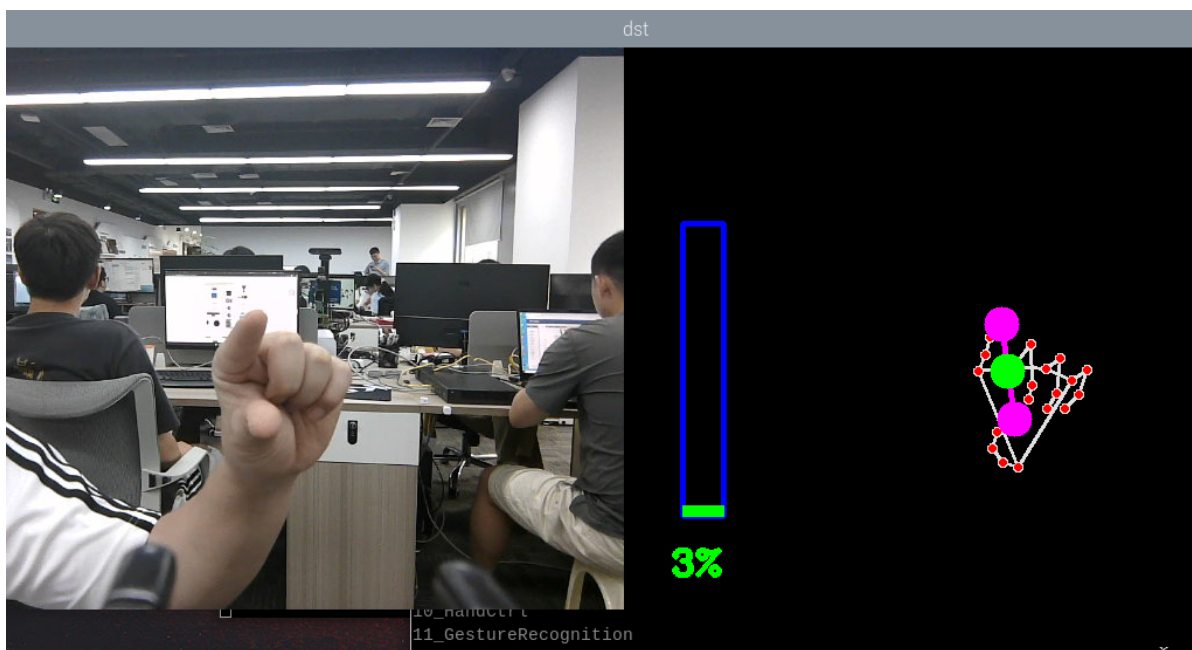
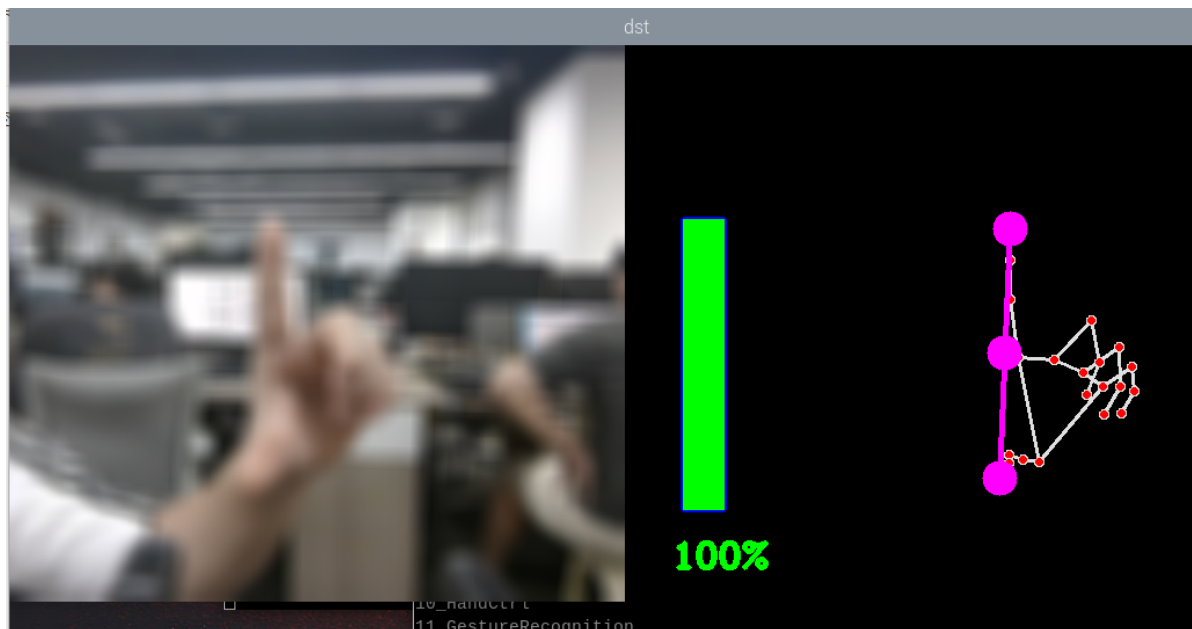
Open a terminal inside the container,

```
exec_m3
```

- Run finger control program node in terminal

```
ros2 launch m3_bringup mediapipe_demo.launch.py demo:=FingerCtrl
```

After the program runs, press the F key to switch image processing effects, then change the distance between the thumb and index finger to change the angle size, thereby controlling the image processing effects.



3. Core Code Analysis

Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/mediapipe/09_FingerCtrl.py
```

Import necessary library files,

```

import math
import time
import cv2 as cv
import numpy as np
import mediapipe as mp
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2

```

Initialize data and define publishers and subscribers,

```

def __init__(self, name):
    super().__init__(name)
    #Define image processing effect list
    self.effect = ["color", "thresh", "blur", "hue", "enhance"]
    self.volBar = 400
    self.pTime = self.cTime = self.volPer = self.value = self.index = 0
    #Use the class in the mediapipe library to define a hand object
    self.mpHand = mp.solutions.hands
    self.mpDraw = mp.solutions.drawing_utils
    self.hands = self.mpHand.Hands(
        static_image_mode=False,
        max_num_hands=2,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5
    )
    #Define the properties of joint connection lines, which will be used in
    subsequent joint point connection functions
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
        thickness=-1, circle_radius=15)
    self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0),
        thickness=10, circle_radius=10)
    self.rgb_bridge = CvBridge()
    #Define the subscriber for color image topic
    self.sub_rgb =
self.create_subscription(Image, "/camera/color/image_raw", self.get_RGBImageCallBa
ck, 100)

```

Color image callback function,

```

def get_RGBImageCallback(self, msg):
    #Use CvBridge to convert color image message data to image data
    frame = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    action = cv2.waitKey(1)
    #Call function to detect hands and draw hand joint connection diagram
    img = self.findHands(frame)
    #Call findPosition to get the coordinates of finger joint list
    lmList = self.findPosition(frame, draw=False)
    if len(lmList) != 0:
        #Calculate angle
        angle = self.calc_angle(4, 0, 8)
        x1, y1 = lmList[4][1], lmList[4][2]
        x2, y2 = lmList[8][1], lmList[8][2]

```

```

cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
cv.circle(img, (x1, y1), 15, (255, 0, 255), cv.FILLED)
cv.circle(img, (x2, y2), 15, (255, 0, 255), cv.FILLED)
cv.line(img, (x1, y1), (x2, y2), (255, 0, 255), 3)
cv.circle(img, (cx, cy), 15, (255, 0, 255), cv.FILLED)
if angle <= 10: cv.circle(img, (cx, cy), 15, (0, 255, 0), cv.FILLED)
#Calculate
self.volBar = np.interp(angle, [0, 70], [400, 150])
self.volPer = np.interp(angle, [0, 70], [0, 100])
self.value = np.interp(angle, [0, 70], [0, 255])
# print("angle: {},value: {}".format(angle, value))
#Perform threshold binarization operation, values greater than threshold
value are represented by 255, values less than threshold value are represented by
0
if self.effect[self.index]=="thresh":
    gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
    frame = cv.threshold(gray, self.value, 255, cv.THRESH_BINARY)[1]
#Perform Gaussian filtering, (21, 21) means the length and width of the
Gaussian matrix are both 21, standard deviation takes value
elif self.effect[self.index]=="blur":
    frame = cv.GaussianBlur(frame, (21, 21), np.interp(self.value, [0, 255],
[0, 11]))
#Color space conversion, HSV to BGR
elif self.effect[self.index]=="hue":
    frame = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
    frame[:, :, 0] += int(self.value)
    frame = cv.cvtColor(frame, cv.COLOR_HSV2BGR)
#Adjust contrast
elif self.effect[self.index]=="enhance":
    enh_val = self.value / 40
    clahe = cv.createCLAHE(clipLimit=enh_val, tileGridSize=(8, 8))
    lab = cv.cvtColor(frame, cv.COLOR_BGR2LAB)
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])
    frame = cv.cvtColor(lab, cv.COLOR_LAB2BGR)
#Press F key to switch processing effects
if action == ord('f'):
    self.index += 1
    if self.index >= len(self.effect): self.index = 0

cv.rectangle(img, (50, 150), (85, 400), (255, 0, 0), 3)
cv.rectangle(img, (50, int(self.volBar)), (85, 400), (0, 255, 0), cv.FILLED)
cv.putText(img, f'{int(self.volPer)}%', (40, 450), cv.FONT_HERSHEY_COMPLEX,
1, (0, 255, 0), 3)
#Merge images
dst = self.frame_combine(frame, img)
cv.imshow('dst', dst)

```

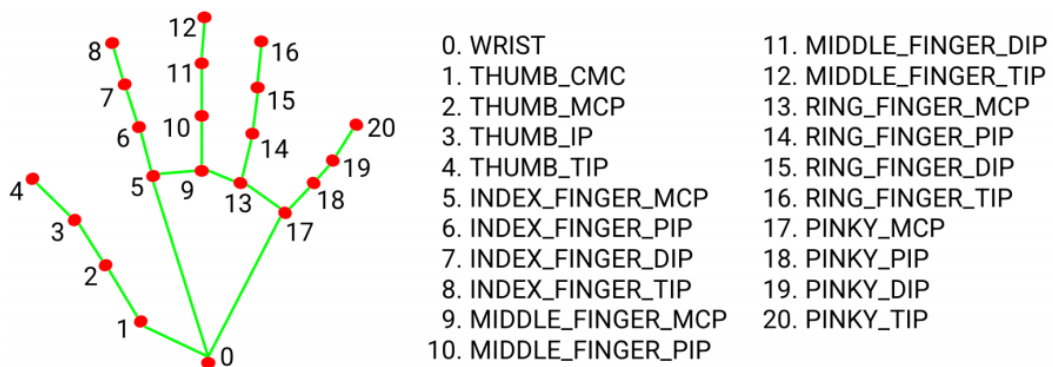
findPosition gets the ID of each joint and the xy coordinates of each joint,

```

def findPosition(self, frame, draw=True):
    #Create detection list to store detection results
    self.lmList = []
    if self.results.multi_hand_landmarks:
        #Iterate through detection results and add detection results to the
        self.lmList list, representing the ID and xy coordinates of each detected joint
        for id, lm in enumerate(self.results.multi_hand_landmarks[0].landmark):
            # print(id,lm)
            h, w, c = frame.shape
            cx, cy = int(lm.x * w), int(lm.y * h)
            # print(id, lm.x, lm.y, lm.z)
            self.lmList.append([id, cx, cy])
            if draw: cv.circle(frame, (cx, cy), 15, (0, 0, 255), cv.FILLED)
    return self.lmList

```

As shown in the figure below, the ID of each finger joint,



calc_angle calculates the angle, here calculating the angle between thumb fingertip, wrist joint, and index fingertip,

```

def calc_angle(self, pt1, pt2, pt3):
    point1 = self.lmList[pt1][1], self.lmList[pt1][2]
    point2 = self.lmList[pt2][1], self.lmList[pt2][2]
    point3 = self.lmList[pt3][1], self.lmList[pt3][2]
    a = self.get_dist(point1, point2)
    b = self.get_dist(point2, point3)
    c = self.get_dist(point1, point3)
    try:
        radian = math.acos((math.pow(a, 2) + math.pow(b, 2) - math.pow(c, 2)) /
(2 * a * b))
        angle = radian / math.pi * 180
    except:
        angle = 0
    return abs(angle)

```